Mini Project 10 — Seleksi Fitur dan Pembekuan Representasi untuk MP11

Mini Project 10 membaca dataset 39-D dan split yang sudah dibekukan pada MP9. Seluruh fit dan pemilihan kandidat dilakukan hanya pada training set. Hasil akhir adalah representasi tetap yang akan digunakan Mini Project 11 untuk membandingkan classifier.

Input dari MP9	Output MP10
features/dataset/mp09_features_39d.npz	features/dataset/mp10_features_final.npz
splits/mp09_split.json	config/feature_reduction_config.json
project_status.json = ready_for_mp10	features/intermediate/mp10_reduction_exploratory.npz
feature_names 39-D	results/metrics/mp10_feature_reduction_metrics.json
train/val/test mask	empat grafik audit dan status ready_for_mp11

Tabel 10.10. Input dan output Mini Project 10.

Tahap	Estimasi
Validasi handoff dan paket	2 menit
Audit variance/korelasi	3 menit
Jalankan kandidat MI, RFE, PCA, LDA	5–7 menit
Periksa grafik dan metrics	4–5 menit
Periksa representasi terpilih dan artefak	3–4 menit
Validasi handoff MP11	2 menit

Tabel 10.11. Rencana waktu Mini Project 10.

Prosedur Step by Step

Langkah 1 — Masuk ke root proyek dan periksa status
Pastikan project_status.json bernilai ready_for_mp10. Jangan membuat split baru dan jangan mengubah mp09_split.json.

Langkah 2 — Verifikasi dependensi
Pastikan NumPy, Matplotlib, dan scikit-learn tersedia. Tidak diperlukan rekaman audio baru.

Langkah 3 — Simpan skrip MP10
Buat mini_project_10.py pada root audio_ai_embedded_project dan salin kode di bawah.

Langkah 4 — Jalankan analisis training-only
Skrip menghitung variance, correlation, mutual information, lalu membandingkan raw39, MI, RFE, PCA, dan LDA menggunakan GroupKFold pada empat acquisition block training.

Langkah 5 — Periksa keputusan hardware-aware
Buka mp10_representation_tradeoff.png dan JSON metrics. Pastikan validation_used_for_selection dan test_used_for_selection bernilai false.

Langkah 6 — Periksa feature transform final
Buka feature_reduction_config.json. Catat chosen_candidate, output_dim, selected_feature_names, scaler_mean, dan scaler_scale.

Langkah 7 — Validasi handoff MP11
Pastikan mp10_features_final.npz ada dan project_status.json berubah menjadi ready_for_mp11.

Kode Python Mini Project 10

```
from pathlib import Path
import json
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.base import clone
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.feature_selection import SelectKBest, mutual_info_classif, RFE
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GroupKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

ROOT = Path('.')
SEED = 2026
F1_TOLERANCE = 0.03
CORR_THRESHOLD = 0.95

# ---------- Helper ----------
def load_json(path):
  with open(path, 'r', encoding='utf-8') as f:
    return json.load(f)

def save_json(path, obj):
  path.parent.mkdir(parents=True, exist_ok=True)
  with open(path, 'w', encoding='utf-8') as f:
    json.dump(obj, f, indent=2, ensure_ascii=False)

def mi_score(X, y):
  return mutual_info_classif(X, y, random_state=SEED)

def cv_macro_f1(pipe, X, y, groups):
  n_groups = len(np.unique(groups))
  cv = GroupKFold(n_splits=n_groups)
  scores = cross_val_score(pipe, X, y, groups=groups, cv=cv, scoring='f1_macro')
  return scores.astype(float)

def transform_selected(X, scaler, indices):
  Xs = scaler.transform(X)
  return Xs[:, indices]

# ---------- 1. Validasi handoff MP9 ----------
status_path = ROOT / 'project_status.json'
status = load_json(status_path)
if status.get('status') != 'ready_for_mp10':
  raise RuntimeError('MP9 belum siap. Status harus ready_for_mp10.')

data_path = ROOT / 'features/dataset/mp09_features_39d.npz'
if not data_path.exists():
  raise FileNotFoundError(data_path)

D = np.load(data_path, allow_pickle=True)
X = D['X'].astype(np.float64)
y = D['y'].astype(str)
groups = D['groups'].astype(str)
sample_ids = D['sample_ids'].astype(str)
feature_names = D['feature_names'].astype(str)
train_mask = D['train_mask'].astype(bool)
val_mask = D['val_mask'].astype(bool)
test_mask = D['test_mask'].astype(bool)

X_train = X[train_mask]
y_train = y[train_mask]
g_train = groups[train_mask]

if X.shape[1] != 39:
  raise RuntimeError(f'Feature dimension diharapkan 39, diperoleh {X.shape[1]}.')
if len(np.unique(g_train)) < 2:
  raise RuntimeError('Training set memerlukan sedikitnya dua group.')

# ---------- 2. Audit fitur pada training set saja ----------
raw_var = np.var(X_train, axis=0, ddof=1)
near_zero = np.where(raw_var < 1e-12)[0]

scaler_audit = StandardScaler().fit(X_train)
Z_train = scaler_audit.transform(X_train)
corr = np.corrcoef(Z_train, rowvar=False)
corr = np.nan_to_num(corr, nan=0.0)
redundant_pairs = []
for i in range(corr.shape[0]):
  for j in range(i + 1, corr.shape[1]):
    if abs(corr[i, j]) >= CORR_THRESHOLD:
      redundant_pairs.append({
        'feature_a': feature_names[i],
        'feature_b': feature_names[j],
```

```
            'abs_corr': float(abs(corr[i, j]))
        })

mi = mi_score(Z_train, y_train)
mi_order = np.argsort(mi)[::-1]

# ---------- 3. Kandidat representasi ----------
base_clf = LogisticRegression(
  solver='lbfgs', max_iter=3000, C=1.0, random_state=SEED
)

k8 = min(8, X.shape[1])
k12 = min(12, X.shape[1])
pca_dim = min(6, X_train.shape[0] - 1, X.shape[1])
lda_dim = min(len(np.unique(y_train)) - 1, 3)

candidates = {
  'raw39': {
    'family': 'raw',
    'dim': int(X.shape[1]),
    'pipeline': Pipeline([
      ('scale', StandardScaler()),
      ('clf', clone(base_clf))
    ])
  },
  'mi8': {
    'family': 'selection',
    'dim': int(k8),
    'pipeline': Pipeline([
      ('scale', StandardScaler()),
      ('select', SelectKBest(mi_score, k=k8)),
      ('clf', clone(base_clf))
    ])
  },
  'mi12': {
    'family': 'selection',
    'dim': int(k12),
    'pipeline': Pipeline([
      ('scale', StandardScaler()),
      ('select', SelectKBest(mi_score, k=k12)),
      ('clf', clone(base_clf))
    ])
  },
  'rfe8': {
    'family': 'selection',
    'dim': int(k8),
    'pipeline': Pipeline([
      ('scale', StandardScaler()),
      ('select', RFE(
        estimator=clone(base_clf),
        n_features_to_select=k8,
        step=0.20
      )),
      ('clf', clone(base_clf))
    ])
  },
  'rfe12': {
    'family': 'selection',
    'dim': int(k12),
    'pipeline': Pipeline([
      ('scale', StandardScaler()),
      ('select', RFE(
        estimator=clone(base_clf),
        n_features_to_select=k12,
        step=0.20
      )),
      ('clf', clone(base_clf))
    ])
  },
  'pca6': {
    'family': 'projection',
    'dim': int(pca_dim),
    'pipeline': Pipeline([
      ('scale', StandardScaler()),
      ('reduce', PCA(n_components=pca_dim, random_state=SEED)),
      ('clf', clone(base_clf))
    ])
  },
  'lda3': {
    'family': 'projection',
    'dim': int(lda_dim),
```

```
      'pipeline': Pipeline([
        ('scale', StandardScaler()),
        ('reduce', LinearDiscriminantAnalysis(n_components=lda_dim, solver='svd')),
        ('clf', clone(base_clf))
      ])
    }
}

metrics = {}
for name, spec in candidates.items():
    scores = cv_macro_f1(spec['pipeline'], X_train, y_train, g_train)
    metrics[name] = {
      'family': spec['family'],
      'dim': spec['dim'],
      'fold_macro_f1': scores.tolist(),
      'mean_macro_f1': float(np.mean(scores)),
      'std_macro_f1': float(np.std(scores))
    }

# ---------- 4. Kebijakan hardware-aware ----------
best_f1 = max(v['mean_macro_f1'] for v in metrics.values())
selection_feasible = [
    name for name, v in metrics.items()
    if v['family'] == 'selection' and v['mean_macro_f1'] >= best_f1 - F1_TOLERANCE
]

if selection_feasible:
    # Prioritas: dimensi lebih kecil, F1 lebih tinggi, MI sebelum RFE jika sama.
    priority = {'mi8': 0, 'rfe8': 1, 'mi12': 2, 'rfe12': 3}
    chosen = sorted(
      selection_feasible,
      key=lambda n: (metrics[n]['dim'], -metrics[n]['mean_macro_f1'], priority.get(n, 99))
    )[0]
else:
    chosen = max(metrics, key=lambda n: metrics[n]['mean_macro_f1'])

# ---------- 5. Fit transform final hanya pada training ----------
final_scaler = StandardScaler().fit(X_train)
Z_all = final_scaler.transform(X)

if chosen.startswith('mi'):
    k = metrics[chosen]['dim']
    scores = mi_score(final_scaler.transform(X_train), y_train)
    selected_idx = np.argsort(scores)[::-1][:k]
    selected_idx = np.sort(selected_idx)
    X_final = Z_all[:, selected_idx]
    final_method = 'mutual_information'
elif chosen.startswith('rfe'):
    k = metrics[chosen]['dim']
    rfe = RFE(estimator=clone(base_clf), n_features_to_select=k, step=0.20)
    rfe.fit(final_scaler.transform(X_train), y_train)
    selected_idx = np.where(rfe.support_)[0]
    X_final = Z_all[:, selected_idx]
    final_method = 'recursive_feature_elimination'
elif chosen.startswith('pca'):
    reducer = PCA(n_components=metrics[chosen]['dim'], random_state=SEED)
    reducer.fit(final_scaler.transform(X_train))
    X_final = reducer.transform(Z_all)
    selected_idx = np.array([], dtype=int)
    final_method = 'pca'
elif chosen.startswith('lda'):
    reducer = LinearDiscriminantAnalysis(n_components=metrics[chosen]['dim'], solver='svd')
    reducer.fit(final_scaler.transform(X_train), y_train)
    X_final = reducer.transform(Z_all)
    selected_idx = np.array([], dtype=int)
    final_method = 'lda'
else:
    X_final = Z_all
    selected_idx = np.arange(X.shape[1])
    final_method = 'standardized_raw'

# Fit exploratory PCA/LDA on training for figures and later comparison only.
pca = PCA(n_components=pca_dim, random_state=SEED).fit(final_scaler.transform(X_train))
lda = LinearDiscriminantAnalysis(n_components=lda_dim, solver='svd').fit(
    final_scaler.transform(X_train), y_train
)

# ---------- 6. Simpan artefak ----------
out_cfg = ROOT / 'config/feature_reduction_config.json'
out_npz = ROOT / 'features/dataset/mp10_features_final.npz'
out_aux = ROOT / 'features/intermediate/mp10_reduction_exploratory.npz'
```

```
out_metrics = ROOT / 'results/metrics/mp10_feature_reduction_metrics.json'
fig_dir = ROOT / 'results/figures'
out_npz.parent.mkdir(parents=True, exist_ok=True)
out_aux.parent.mkdir(parents=True, exist_ok=True)
out_metrics.parent.mkdir(parents=True, exist_ok=True)
fig_dir.mkdir(parents=True, exist_ok=True)

selected_names = feature_names[selected_idx].tolist() if len(selected_idx) else []

config = {
  'source_dataset': 'features/dataset/mp09_features_39d.npz',
  'fit_scope': 'training_only',
  'cv_strategy': 'GroupKFold over MP9 training acquisition blocks',
  'seed': SEED,
  'correlation_threshold': CORR_THRESHOLD,
  'f1_tolerance_for_hardware_aware_selection': F1_TOLERANCE,
  'chosen_candidate': chosen,
  'final_method': final_method,
  'input_dim': int(X.shape[1]),
  'output_dim': int(X_final.shape[1]),
  'selected_indices': selected_idx.tolist(),
  'selected_feature_names': selected_names,
  'scaler_mean': final_scaler.mean_.tolist(),
  'scaler_scale': final_scaler.scale_.tolist(),
  'validation_used_for_selection': False,
  'test_used_for_selection': False
}
save_json(out_cfg, config)

np.savez_compressed(
  out_npz,
  X=X_final.astype(np.float32), y=y, groups=groups,
  sample_ids=sample_ids,
  train_mask=train_mask, val_mask=val_mask, test_mask=test_mask,
  source_feature_names=feature_names,
  selected_indices=selected_idx,
  selected_feature_names=np.array(selected_names, dtype=object),
  method=np.array([final_method], dtype=object)
)

np.savez_compressed(
  out_aux,
  corr=corr.astype(np.float32),
  mutual_information=mi.astype(np.float32),
  mi_order=mi_order.astype(np.int32),
  pca_components=pca.components_.astype(np.float32),
  pca_explained_variance_ratio=pca.explained_variance_ratio_.astype(np.float32),
  lda_scalings=lda.scalings_.astype(np.float32)
)

report = {
  'n_samples_total': int(len(X)),
  'n_train': int(train_mask.sum()),
  'n_validation': int(val_mask.sum()),
  'n_test': int(test_mask.sum()),
  'n_features_input': int(X.shape[1]),
  'near_zero_variance_features': feature_names[near_zero].tolist(),
  'n_redundant_pairs_abs_corr_ge_threshold': len(redundant_pairs),
  'top_10_mutual_information': [
    {'feature': feature_names[i], 'score': float(mi[i])}
    for i in mi_order[:10]
  ],
  'candidate_cv': metrics,
  'best_cv_macro_f1': float(best_f1),
  'chosen_candidate': chosen,
  'chosen_output_dim': int(X_final.shape[1]),
  'selected_feature_names': selected_names,
  'validation_used_for_selection': False,
  'test_used_for_selection': False
}
save_json(out_metrics, report)

# ---------- 7. Visualisasi ----------
# A. Correlation heatmap
plt.figure(figsize=(8.5, 7.0))
plt.imshow(corr, vmin=-1, vmax=1, aspect='auto', cmap='coolwarm')
plt.colorbar(label='Pearson correlation')
plt.title('MP10 - Korelasi fitur pada training set')
plt.xlabel('Feature index')
plt.ylabel('Feature index')
plt.tight_layout()
```

```
plt.savefig(fig_dir / 'mp10_correlation_matrix.png', dpi=170)
plt.close()

# B. MI top-15
n_show = min(15, len(feature_names))
idx_show = mi_order[:n_show][::-1]
plt.figure(figsize=(8.5, 5.8))
plt.barh(np.arange(n_show), mi[idx_show])
plt.yticks(np.arange(n_show), feature_names[idx_show], fontsize=8)
plt.xlabel('Mutual information score')
plt.title('MP10 - Ranking fitur training-only')
plt.tight_layout()
plt.savefig(fig_dir / 'mp10_feature_ranking.png', dpi=170)
plt.close()

# C. Candidate comparison
cand_names = list(metrics.keys())
means = [metrics[n]['mean_macro_f1'] for n in cand_names]
dims = [metrics[n]['dim'] for n in cand_names]
plt.figure(figsize=(8.5, 5.4))
plt.scatter(dims, means, s=65)
for n, x0, y0 in zip(cand_names, dims, means):
    plt.annotate(n, (x0, y0), xytext=(4, 4), textcoords='offset points', fontsize=8)
plt.xlabel('Dimensi representasi')
plt.ylabel('Grouped-CV macro-F1 (training only)')
plt.ylim(max(0, min(means)-0.1), min(1.03, max(means)+0.05))
plt.title('MP10 - Trade-off performa dan dimensi')
plt.grid(alpha=0.25)
plt.tight_layout()
plt.savefig(fig_dir / 'mp10_representation_tradeoff.png', dpi=170)
plt.close()

# D. PCA projection training
Ztr = final_scaler.transform(X_train)
P2 = PCA(n_components=2, random_state=SEED).fit_transform(Ztr)
plt.figure(figsize=(7.5, 5.6))
for cls in np.unique(y_train):
    m = y_train == cls
    plt.scatter(P2[m, 0], P2[m, 1], label=cls, s=55)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('MP10 - Proyeksi PCA 2D training set')
plt.legend()
plt.grid(alpha=0.2)
plt.tight_layout()
plt.savefig(fig_dir / 'mp10_pca_projection.png', dpi=170)
plt.close()

status['status'] = 'ready_for_mp11'
status['last_completed_mini_project'] = 10
status['mp10'] = {
    'chosen_candidate': chosen,
    'final_method': final_method,
    'output_dim': int(X_final.shape[1]),
    'feature_reduction_config': str(out_cfg),
    'final_feature_dataset': str(out_npz),
    'metrics': str(out_metrics)
}
save_json(status_path, status)

print('Mini Project 10 selesai.')
print('Chosen representation:', chosen)
print('Output dimension:', X_final.shape[1])
print('Status:', status['status'])
```

Kriteria Lulus Mini Project 10

Pemeriksaan	Kriteria lulus
Status awal	ready_for_mp10
Fit scope	training_only
CV	GroupKFold menggunakan acquisition block training
Validation/test	Tidak dipakai untuk selection
Representasi akhir	output_dim < 39 atau raw39 dipilih dengan alasan eksplisit
Config	feature_reduction_config.json menyimpan scaler dan transform
Dataset akhir	mp10_features_final.npz memiliki 24 sampel dan mask split yang sama

Pemeriksaan	Kriteria lulus
Status akhir	ready_for_mp11

Tabel 10.12. Checklist kelulusan Mini Project 10.

Artefak yang Wajib Dibawa ke Mini Project 11

- `features/dataset/mp10_features_final.npz` — representasi final semua sampel dengan split mask yang tetap.
- `config/feature_reduction_config.json` — mean, scale, metode, indeks/nama fitur atau parameter proyeksi.
- `results/metrics/mp10_feature_reduction_metrics.json` — grouped-CV comparison dan audit leakage.
- `features/intermediate/mp10_reduction_exploratory.npz` — korelasi, MI, PCA, dan LDA untuk analisis.
- `splits/mp09_split.json` — tetap menjadi sumber kebenaran split; tidak diubah.
- `project_status.json` dengan status `ready_for_mp11`.

Mini Project 11 tidak melakukan feature selection ulang. Tugasnya adalah membandingkan classifier pada representasi yang sudah dibekukan di sini, sehingga perbedaan performa dapat diatribusikan lebih jelas pada model, bukan pada perubahan pipeline fitur.

Referensi Bab

1. Bishop, C. M. Pattern Recognition and Machine Learning. Springer.

2. Hastie, T., Tibshirani, R., dan Friedman, J. The Elements of Statistical Learning. Springer.

3. Guyon, I., dan Elisseeff, A. An Introduction to Variable and Feature Selection. Journal of Machine Learning Research.

4. Jolliffe, I. T., dan Cadima, J. Principal Component Analysis: A Review and Recent Developments. Philosophical Transactions of the Royal Society A.

5. Fisher, R. A. The Use of Multiple Measurements in Taxonomic Problems. Annals of Eugenics.

6. Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly.

7. Scikit-learn documentation: feature selection, decomposition, discriminant analysis, pipelines, dan model selection.